

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 5

CSS Layout-Frameworks

5.1 CSS Flexible Box Layout

In Vorlesung 4 wurden mehrere Layout-Modelle erwähnt:

- Block-Layout (`display: block`), mit dem Dokumente standardmäßig dargestellt werden,
- Inline-Layout (`display: inline`), mit dem Elemente auf Textebene angeordnet werden,
- Position-Layout (`position`) für explizite Positionierungen und
- Tabellen-Layout (`display: table`), welches die Tabellendarstellung regelt (aber nicht im Detail vorgestellt wurde).

Das *CSS Flexible Box Layout* [14], kurz Flexbox stellt nun ein weiteres Modell – nämlich Flex-Layout – dar. Es behebt einige grundsätzliche Probleme von den zuvor erwähnten Modellen und auch dem eigentlich zweckentfremdeten Floating. Zunächst lassen sich Boxen einfach nebeneinander positionieren, ohne auf `float` zurückgreifen zu müssen. Diese Boxen nützen selbst den vorhandenen Platz aufgrund zugewiesener Verhältnisse aus und passen sich auch gleichmäßig in der Höhe an, wenn ein Element etwa mehr Text enthält als ein anderes.

5.1.1 Einen flexiblen Container definieren

Die prinzipielle Arbeitsweise mit flexiblen Boxen sieht so aus: ein Container mit mehreren Kindelementen enthält die Eigenschaft `display: flex`, wodurch das Flex-Layout aktiviert wird. Dieser Container verbreitert seine Inhalte, um freien Platz zu nützen oder schrumpft sie, um einen Text-Überfluss zu verhindern. Der verwendete Layout-Algorithmus ist dabei unabhängig von Richtungen (im Gegensatz zu Block-Elementen die vertikal-lastig und Inline-Elementen, die horizontal-lastig sind).

Die Eigenschaft `flex-direction` bestimmt, ob Boxen nebeneinander oder untereinander platziert werden. Dies kann durch die Werte `row` oder `column` (bzw. `row-reverse` oder `column-reverse`) festgelegt werden. Mittels `flex-wrap` und den Werten `wrap` (bzw. `wrap-reverse`) oder `nowrap` kann dann noch bestimmt werden, ob ein flexibler Container ein- oder mehrzeilig werden darf. `flex-flow` ist ein Shorthand-Property und nimmt zwei Angaben für die Richtung und das Umbruchsverhalten.

Im folgenden Beispiel wird ein dreispaltiges Layout mit flexiblen Boxen erstellt. Als Grundlage dient das folgende HTML-Grundgerüst. Innerhalb eines `<main>`-Elements befinden sich ein `<section>`-Element für den Inhalt, ein `<nav>`-Element für die Navigation und ein `<aside>`-Element mit Zusatzinformationen in einer Art Sidebar. Die Anordnung der inneren Elemente (`<nav>` zwischen `<article>` und `<aside>`) wurde bewusst so gewählt, um eine mögliche (und mit Flexbox einfache) Neuordnung in CSS demonstrieren zu können.

```
1 <body>
2   <div id="container">
3     <header>
4       <h1>Die Überschrift</h1>
5     </header>
6     <main>
7       <section id="content">Lorem ipsum...</section>
8       <nav>
9         <ul>
10          <li>Navigation 1</li>
11          <li>Navigation 2</li>
12        </ul>
13      </nav>
14      <aside>Lorem ipsum...</aside>
15    </main>
16    <footer>&copy; 2025 Der*Die Autor*in</footer>
17  </div>
18 </body>
```

Das `<main>`-Element dient als Flexbox-Container und erhält die beiden Eigenschaften `display: flex` und `flex-direction: row`. Dadurch werden die drei Gliederungselemente innerhalb nebeneinander angezeigt. Die Angabe von `min-height` garantiert eine Mindesthöhe der Elemente. Ansonsten wären diese so hoch wie das höchste der drei Elemente. Der Container, der alles umgibt, wird mit 960 Pixeln Breite definiert.

```
1 * {
2   margin: 0;
3   box-sizing: border-box;
4 }
5 body {
6   font-family: Raleway, Arial, sans-serif;
7   color: rgb(236 233 230);
8   line-height: 2rem;
9   hyphens: auto; /* Silbentrennung */
10 }
11 div#container {
12   width: 960px;
13   margin: 0 auto;
14   background-color: rgb(255 132 0);
15 }
16 main {
17   display: flex;
18   flex-direction: row;
19   min-height: 25rem;
20 }
```

5.1.2 Flexibilität und Reihenfolge der Boxen festlegen

Mit dem Schlüsselwort **flex** wird nun festgelegt, wie sich die im flexiblen Container enthaltenen Elemente verhalten. Erwartet werden drei Angaben: die erste definiert die Wachstumsrate im Bezug zu den anderen, die zweite die Schrumpfrate, der dritte Wert bestimmt die Basisgröße. **flex: 1 1 auto** bedeutet, dass ein Element gleichmäßig wächst und schrumpft, **flex: 5 1 auto** lässt ein Element im Vergleich zu den anderen 5 Mal so stark anwachsen. Da in beiden Fällen der dritte Wert mit **auto** festgelegt ist, nehmen die Spalten von Beginn an die gleiche Breite ein. Alternativ können etwa auch Prozentangaben angegeben werden: gibt es zwei Container mit den Angaben **flex: 3 1 30%** und **flex: 1 1 30%**, so stehen beiden Containern jeweils 30 % der Gesamtbreite zu. De facto wird der erste Container von den nicht aufgeteilten 40% jedoch drei Mal so viel beanspruchen wie der zweite, weshalb der erste Container 60 % und der zweite Container 40 % der Gesamtbreite einnehmen werden. Eine **flex**-Angabe von 0 bedeutet, dass das Element gar nicht flexibel ist, also seine ursprüngliche Größe immer beibehalten wird.

flex ist das Shorthand-Property für **flex-grow**, **flex-shrink** und **flex-basis**, könnte also auch mit drei separaten CSS-Eigenschaften geschrieben werden.

Über die Eigenschaft **order** kann die Reihenfolge der Darstellung geändert werden, es kommen Zahlenwerte von 1 bis n zum Einsatz.

Im Beispiel teilen sich die drei Elemente ausgehend von den 960 Pixeln Breite des Containers auf 600px (entspricht 62,5 %), 240px (25 %) und 120px (12,5 %) auf, somit sind 100 % erreicht und es ist bei dieser Breite kein Platz zum Wachsen oder Schrumpfen. Der Abschnitt darf aber beim Wachsen doppelt so viel Platz in Anspruch nehmen wie Navigation und Seitenleiste. Beim Schrumpfen verliert er aber auch anteilmäßig doppelt so viel Platz wie die anderen beiden Elemente. Durch die Angabe von **order** wird die Navigation ganz links, der Artikel in der Mitte und die Sidebar ganz rechts dargestellt.

```

1 main > section {
2   flex: 2 2 600px; /* 62,5 % von 960px */
3   order: 2;
4   color: rgb(67 67 67);
5   background-color: rgb(255 182 0);
6 }
7 main > nav {
8   flex: 1 1 240px; /* 25 % von 960px */
9   order: 1;
10  background-color: rgb(255 132 0);
11 }
12 main > aside {
13   flex: 1 1 120px; /* 12,5 % von 960px */
14   order: 3;
15   background-color: rgb(228 42 135);
16 }
```

Wachsen von Elementen

Der Container ist grundsätzlich hier mit einer Breite von 960 Pixeln festgelegt. Auch die Basisbreite der Elemente ist genau auf diese Dimension ausgerichtet. Die folgende Media Query legt jedoch fest, dass ab 1200 Pixeln Fensterbreite auch der Container auf

dieselbe Breite gesetzt wird:

```
1 @media all and (min-width: 1200px) {
2   div#container {
3     width: 1200px;
4   }
5 }
```

Somit stehen für die drei mit Flexbox angeordneten Elemente nun nicht mehr 960 Pixel, sondern 1200 Pixel, also um 240 Pixel mehr Bereite zur Verfügung. Diese werden nun anteilmäßig aufgeteilt. Die Verhältnisse sind mit 2 zu 1 zu 1 in der ersten `flex`-Eigenschaft definiert, d. h. es gibt 4 Anteile, die zu vergeben sind. Mit folgender Formel lässt sich nun der neu gewonnene Platz berechnen:

$$width_{new} = \left(\frac{flex-grow}{flex-grow_{total}} \cdot free-space \right) \cdot width$$

Für das Abschnitt-Element ist der *flex-grow* Wert 2, für die anderen beiden jeweils 1, *flex-grow_{total}* ist also 4 und *free-space* ist 240. *width* ist die jeweils ursprüngliche Breite, also 600, 240 und 120 Pixel. Somit ergeben sich folgende neue Breiten:

- Abschnitt: $width_{new} = \left(\frac{2}{4} \cdot 240 \right) \cdot 600 = 720$
- Navigation: $width_{new} = \left(\frac{1}{4} \cdot 240 \right) \cdot 240 = 300$
- Seitenleiste: $width_{new} = \left(\frac{1}{4} \cdot 240 \right) \cdot 120 = 180$

Anders ausgedrückt, die 240 Pixel Platz verteilen sich im Verhältnis 2:1:1, also 50 %, 25 % und 25 % auf die drei Elemente, was einen Platzgewinn von 120, 60 und 60 Pixeln bedeutet.

Schrumpfen von Elementen

Das Schrumpfen von Elementen ist etwas komplizierter zu berechnen, da nicht nur der neu gewonnene Platz aufgeteilt werden muss, sondern das Schrumpfen im Zusammenhang mit der bisherigen Breite (`flex-shrink`) steht. Um den Container unter 960 Pixel schrumpfen zu lassen, ist folgende Media Query verantwortlich. Sie setzt den Container auf die volle Browserfensterbreite:

```
1 @media all and (max-width: 960px) {
2   div#container {
3     width: 100%;
4   }
5 }
```

Wird dieses nun auf 720 Pixel verkleinert, entsteht die entsprechende Containerbreite, somit fehlen nun 240 Pixel verglichen zur ursprünglichen Größe. Um nun die Schrumpfung der drei Elemente zu berechnen, muss zuerst die Schrumpfrate ermittelt werden:

$$shrink-ratio = \frac{width \cdot flex-shrink}{shrink-scaled-width_{total}}$$

shrink-scaled-width_{total} ist die Summe aller Breiten multipliziert mit der jeweiligen `flex-shrink`-Angabe. Hier also $600 \cdot 2 + 240 \cdot 1 + 120 \cdot 1 = 1560$. Somit ergeben sich die folgenden Schrumpfraten:

- Abschnitt: $shrink-ratio = \frac{600 \cdot 2}{1560} = 0,7693$
- Navigation: $shrink-ratio = \frac{240 \cdot 1}{1560} = 0,1538$
- Seitenleiste: $shrink-ratio = \frac{120 \cdot 1}{1560} = 0,0769$

Somit ist berechnet, wie stark ein Element schrumpft. Nun kann seine neue Breite ausgerechnet werden:

$$width_{new} = width - (shrink-space \cdot shrink-ratio)$$

Mit den Schrumpfraten von oben, den ursprünglichen Breiten ($width$) von 600, 240 und 120 Pixeln sowie dem fehlenden Platz ($shrink-space$) von 240 Pixeln ergeben sich die folgenden neuen Breiten:

- Abschnitt: $width_{new} = 600 - (240 \cdot 0,7693) = 415,38$
- Navigation: $width_{new} = 240 - (240 \cdot 0,1538) = 203,08$
- Seitenleiste: $width_{new} = 120 - (240 \cdot 0,0769) = 101,54$

Beim Schrumpfen werden also die fehlenden 240 Pixel nicht im Verhältnis 2:1:1 aufgeteilt, vielmehr spielt die ursprüngliche Breite bzw. das ursprüngliche Verhältnis hier eine Rolle bei der Berechnung. Ebenfalls können die Werte abweichen, wenn sich in den Boxen Inhalt (Text) befindet, der das Schrumpfen etwa bei nicht möglichen Zeilenumbrüchen behindert.

5.1.3 Ausrichtung der Boxen bestimmen

Ist ein Container größer (d. h. breiter oder höher) als die Boxen darin (da diese etwa über fixe Angaben in ihrer Größe beschränkt sind) können die Boxen einfach angeordnet werden. Hier zeigt sich der große Vorteil der flexiblen Boxen gegenüber den bisherigen Modellen, da mit zwei Anweisungen die verschiedensten horizontalen und vertikalen Platzierungen erreicht werden können. Die Angaben für die Ausrichtung werden direkt beim Flex-Container gesetzt.

Mit **justify-content** kann die Ausrichtung der Boxen an der Hauptachse bestimmt werden. Diese ist durch **flex-flow** vorgegeben. Ist der Wert **row**, so ist die Hauptachse in der Horizontalen, ist der Wert **column** ist sie vertikal angeordnet. Es gibt daher auch nur die Angaben **flex-start** und **flex-end** (und nicht etwa **left**, **right** oder **top**, **bottom**). Ersteres richtet die Elemente am Beginn, zweiteres am Ende der Hauptachse aus. Ist die Hauptachse also horizontal, bedeutet **flex-start** linksbündig, ist sie vertikal bedeutet es oben ausgerichtet, **flex-end** bedeutet damit rechtsbündig bzw. unten ausgerichtet. Weiters existieren noch die Werte **center** (zentriert), **space-between** (äußerste Elemente am Rand, Platz dazwischen gleichmäßig verteilt), **space-around** (jedes Element hat links und rechts gleich viel Abstand) und **space-evenly** (der Abstand zwischen allen Elementen und dem Rand ist gleich).

align-items entspricht der Ausrichtung von Elementen auf der Querachse (die rechtwinklig zur Hauptachse steht). Hier gibt es die Werte **flex-start** (oben/links), **flex-end** (unten/rechts), **center** (zentriert), **stretch** (gestreckt über die gesamte Höhe) und **baseline** (Elemente sind an einer Grundlinie ausgerichtet).

Existieren auf der Querachse mehrere Reihen an Boxen, können diese mit der Eigenschaft **align-content** ausgerichtet werden. Die Werte gleichen denen für **align-items**,

bei `flex-start` etwas werden alle Reihen ganz oben angeordnet, bei `center` werden sie zentriert usw.

Mit der Eigenschaft `gap` kann ein gleichmäßiger Abstand zwischen den Boxen in beiden Richtungen erzielt werden, mit `row-gap` und `column-gap` je auf einer Achse.

Sollte die Ausrichtung eines einzelnen Elements anders als bei den übrigen im Container gewünscht sein, so kann mittels `align-self` diese Angabe direkt beim Element gemacht werden und die Werte des Containers überschreiben.

Im Beispiel ist nun auch das `<header>`-Element ein Flex-Container und richtet die darin enthaltene Überschrift mit einer Anweisung vertikal zentriert aus. Im Footer wird der Text rechts unten durch eine zweimalige Angabe von `flex-end` bei `align-items` und `justify-content` platziert.

```
1 header {
2   display: flex;
3   align-items: center;
4   background-color: rgb(250 94 94);
5 }
6 footer {
7   display: flex;
8   align-items: flex-end;
9   justify-content: flex-end;
10  background-color: rgb(67 67 67);
11 }
12 header, footer {
13   min-height: 6rem;
14 }
```

Selbstverständlich kann auch hier responsive gearbeitet werden. Die folgenden Media Queries erwirken einen fluiden (d. h. 100 Prozent breiten) Container unter 960 Pixeln und eine Untereinanderreihung der Spalten bei Displays, die schmaler als 640 Pixel sind. Dazu wird die Richtung der flexiblen Boxen von Reihe auf Spalte gewechselt. Zudem wird die ursprüngliche Ordnung der Elemente wiederhergestellt und `flex-basis` wird auf `auto` gestellt, denn ansonsten würden die Pixelangaben jetzt für die Höhe gelten, was nicht gewünscht wäre:

```
1 @media all and (max-width: 960px) {
2   div#container {
3     width: 100%;
4   }
5 }
6 @media all and (max-width: 640px) {
7   main {
8     flex-direction: column;
9   }
10  main > section, main > nav, main > aside {
11    order: 0;
12    flex-basis: auto;
13  }
14 }
```

Abbildung 5.1 zeigt dieses Layout mit 960er, 1200er, 720er und 600er Containerbreite.

Eine sehr gute Einführung zum Thema Flexible Box mit allen Begriffen und bildlichen Erklärungen bietet [3].



Abbildung 5.1: Bei 960 Pixeln ist die Spaltenbreite bei 240 für die Navigation, 600 für den Abschnitt und 120 für die Seitenleiste (a). Bei 1200 Pixeln wachsen die Spalten wie beschrieben auf 300, 720 und 180 Pixel (b). Bei 720 Pixeln schrumpfen sie auf 203,08, 415,38 und 101,54 Pixel (c) und unter 640 Pixeln werden sie untereinander angeordnet und die Reihenfolge wird geändert (d).

5.2 CSS Grid Layout

Elemente an einem Raster auszurichten ist ein üblicher Prozess in der Gestaltung, sowohl für den Druck- als auch den Webbereich. In der Vergangenheit existierten jedoch für den Webbereich nur unzulängliche Lösungen, dies zu erreichen. Layouts mittels `float`-Anweisungen funktionieren prinzipiell, haben jedoch das Problem, dass im normalen Darstellungsmodell Boxen nur so hoch sind, wie ihr Inhalt, was für mehrspaltige Layouts problematisch ist und Workarounds bedarf (die z. B. in verschiedensten CSS-Frameworks fast bis zur Perfektion getrieben wurden). Das Flexbox-Layout löst dieses Problem, allerdings ist es per se nur ein eindimensionales Layout, d. h. Elemente werden nur in einer Zeile oder Spalte angeordnet und brechen einfach um. Die Unzulänglichkeiten hierbei zeigen sich etwa, wenn sieben flexible Boxen so formatiert werden, dass sich drei in einer Zeile ausgehen. Es entsteht ein dreispaltiges Layout, wobei die ersten zwei Zeilen jeweils drei Boxen enthalten, die dritte Zeile jedoch nur eine, welche die gesamte Breite beansprucht. Auch dies ist wiederum nur mit Workaround zu lösen.

Das *CSS Grid Layout* [15] löst all diese Probleme, indem es ermöglicht, ein zweidimensionales Raster in CSS zu erstellen. Die grundsätzliche Vorgangsweise ist dabei ähnlich wie bei Flexbox: ein Element wird zum Container gemacht (hier der Grid-Container), dieser definiert die grundlegenden Eigenschaften (hier z. B. Reihen und Spalten) die darin enthaltenen Elemente werden darin angeordnet.

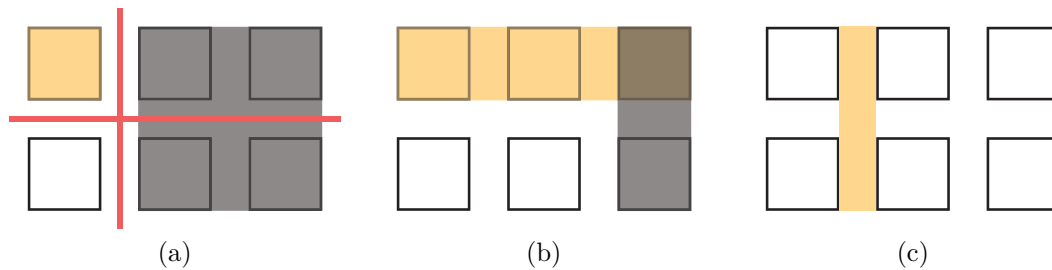


Abbildung 5.2: Terminologie im Grid. (a): Grid Cell (gelb), Grid Lines (rot), Grid Area (grau). (b): Grid Tracks: Grid Line (gelb) und Grid row (grau). (c): Gutter.

5.2.1 Grid Terminologie

Vor der Arbeit mit dem Grid Layout sollen einige Begriffe zu definiert werden, um das Verständnis zu erleichtern:

- **Grid Lines** (Rasterlinien): Die vertikalen und horizontalen Linien, die den Raster unterteilen und die Reihen und Spalten voneinander trennen (Abbildung 5.2 (a)).
- **Grid Cell** (Rasterzelle): Eine einzelne Einheit im CSS Grid (Abbildung 5.2 (a)).
- **Grid Area** (Rasterbereich): Ein rechteckiger Bereich, bestehend aus mehreren Rasterzellen, der von vier Rasterlinien begrenzt wird (Abbildung 5.2 (a)).
- **Grid Track** (Rasterspur): Der Bereich zwischen zwei Rasterlinien, entweder horizontal oder vertikal (Abbildung 5.2 (b)).
- **Grid Row** (Rasterzeile): Eine horizontale Rasterspur (Abbildung 5.2 (b)).
- **Grid Column** (Rasterspalte): Eine vertikale Rasterspur (Abbildung 5.2 (b)).
- **Gutter** (Steg): Der Zwischenraum zwischen Reihen und Spalten in einem Grid (Abbildung 5.2 (c)).
- **Grid Container:** Das Element, welches das gesamte Grid beinhaltet.
- **Grid Item:** Jedes direkte Kindelement eines Grid Containers.

5.2.2 Einen Grid-Container definieren

Zunächst wird ein Element bestimmt, das als Grid-Container agiert. Es beinhaltet das gesamte Grid und wird mit der Eigenschaft `display: grid` als solches festgelegt. Alternativ kann mit `display: inline-grid` auch ein Grid auf Textebene definiert werden.

Aufteilung des Grids

Für dieses Element wird nun die grundsätzliche Aufteilung des Rasters definiert. Mit den Eigenschaften `grid-template-columns` und `grid-template-rows` werden die Spalten und Zeilen definiert. Beide Eigenschaften akzeptieren durch Leerzeichen getrennte Angaben. `grid-template-columns: 150px 150px 150px` etwa erzeugt drei Spalten mit jeweils 150 Pixeln Breite.

Mit dem Schlüsselwort `auto` kann der verbleibende Platz zwischen fixen Angaben automatisch bzw. flexibel vergeben werden, so erzeugt z. B. `grid-template-rows: 200px`

`auto 200px` die erste und dritte Reihe mit 200 Pixeln Höhe während die zweite den restlichen, im Container verbleibenden Platz einnimmt.

Soll der freie, verbleibende Platz in einem Verhältnis aufgeteilt werden, kann die extra für das Grid eingeführte Einheit `fr` (für Fraction = Anteil) verwendet werden. `grid-template-columns: 1fr 1fr 1fr` erzeugt drei Spalten, die sich jeweils zu gleichen Anteilen den Platz im Container aufteilen.

Um unnötiges Wiederholen derartiger Angaben zu vermeiden, kann die `repeat()`-Syntax angewandt werden. Das Beispiel mit den drei anteilsmäßig gleich verteilten Spalten lässt sich auch folgendermaßen schreiben: `grid-template-columns: repeat(3, 1fr)`.

Auto-Placement von Elementen im Grid und Subgrid

Sobald ein Grid definiert (aufgeteilt) ist, werden die direkten Kindelemente des Grid-Containers darin platziert. Dieses Verhalten nennt sich *Auto-Placement* und ordnet pro Grid Cell ein Element an. Werden nur Spalten mit `grid-template-columns` definiert, werden dabei so viele Zeilen erzeugt, wie nötig.

Im folgenden Beispiel wird ein `<div>`-Element zum Grid-Container mit zwei Spalten. Die darin verschachtelten `<div>`-Elemente werden durch Auto-Placement in die Grid-Zellen gelegt.

```
1 <p>Der folgende Container nimmt standardmäßig die volle verfügbare Breite ein:</p>
2
3 <div class="container block-grid">
4   <div class="item">A</div>
5   <div class="item">B</div>
6 </div>
7
8 <p>Dadurch wird der Textfluss vor und nach dem Grid unterbrochen.</p>
```

Das Grid wird nur durch diesen Code erzeugt, in der Klasse `.item` werden nur optische Formatierungen gemacht, keine explizite Platzierung (daher Auto-Placement). Das Ergebnis ist unter Punkt 1 in Abbildung 5.3 zu sehen.

```
1 .block-grid {
2   display: grid;
3   grid-template-columns: 1fr 1fr; /* Zwei gleich große Spalten */
4   gap: 0.625rem;
5 }
6 .item {
7   background-color: rgb(255 204 0);
8   padding: 0.625rem;
9   text-align: center;
10  font-weight: bold;
11 }
```

Ebenso kann ein Grid in einem Inline-Element erstellt werden. Dafür wird die Eigenschaft `display: inline-grid` gesetzt. Der Rest funktioniert identisch. Das folgende Grid wird in einem `<span>`-Element angelegt, funktioniert ansonsten aber gleich wie ein Block-Grid. Abbildung 5.3 zeigt das Inline-Grid unter Punkt 2.

```
1 <p>
2   Hier ist ein Fließtext, und mittendrin befindet sich
3   <span class="container inline-grid">
```

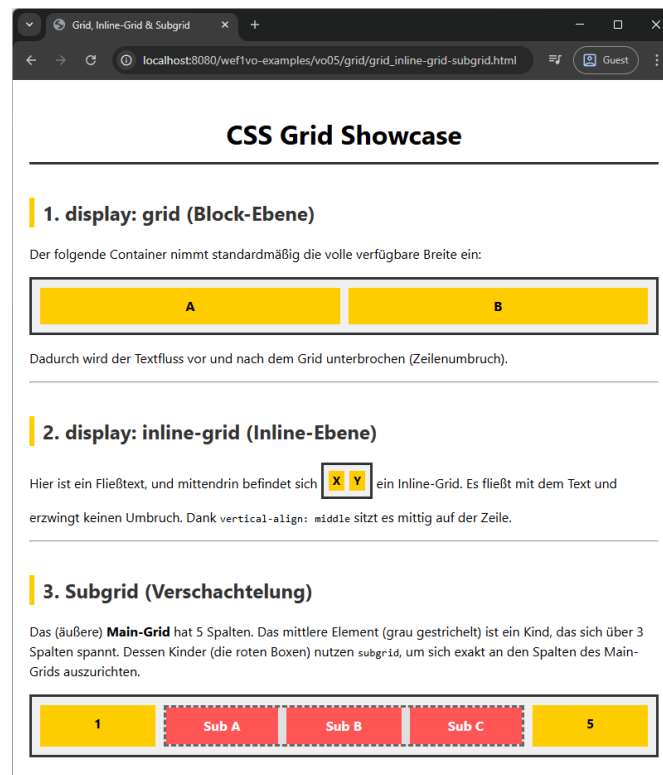


Abbildung 5.3: Teil 1 zeigt ein Grid mit `display: grid` als Block-Element. Teil 2 zeigt ein Inline-Grid mit `display: inline-grid` und in Teil 3 wird ein Subgrid mit `display: grid` und `grid-template-columns: subgrid` dargestellt.

```

4   <span class="item">X</span>
5   <span class="item">Y</span>
6   </span>
7   ein Inline-Grid. Es fließt mit dem Text und erzwingt keinen Umbruch.
8 </p>

```

Beim Inline-Grid macht die Spaltenbreite mit dem Wert `auto` Sinn, damit sie sich dem Inhalt anpassen. Die Grid-Items werden auch hier automatisch platziert.

```

1 .inline-grid {
2   display: inline-grid;
3   grid-template-columns: auto auto; /* Spalten passen sich Inhalt an */
4   gap: 0.4rem;
5   vertical-align: middle; /* Zentriert das Grid vertikal zur Textzeile */
6   padding: 0.4rem;
7 }
8 .inline-grid .item {
9   padding: 0.1rem 0.3rem;
10  font-size: 0.9em;
11 }

```

Schließlich ist es mit dem CSS Grid Layout Module Level 2 [16] auch möglich, Subgrids zu definieren. War es zunächst nur vorgesehen, die direkten Kindelemente des Grid-Containers im Grid anzuordnen, kann das Grid nun auch an weiter „unten“ liegen-

de Elemente vererbt werden. So kann mit `display: grid` ein weiteres Grid innerhalb eines bestehenden erzeugt werden und diese durch `grid-template-columns: subgrid` angewiesen werden, die Spaltenaufteilung des äußeren Grids zu übernehmen.

Das folgende Beispiel legt ein äußeres Grid mit 5 Spalten an. Das 1. und 5. Element werden direkt durch dieses Grid angeordnet. Das zweite Element erstreckt sich über drei Spalten, ist selbst wieder ein Grid übernimmt die Einteilung des äußeren Grids. Das Ergebnis ist in Teil 3 von Abbildung 5.3 zu sehen.

```
1 <div class="container main-grid">
2   <div class="item">1</div>
3   <div class="subgrid-container">
4     <div class="sub-item">Sub A</div>
5     <div class="sub-item">Sub B</div>
6     <div class="sub-item">Sub C</div>
7   </div>
8   <div class="item">5</div>
9 </div>
```

Die Klasse `.sub-item` dient hier auch nur zur Formatierung. Die Elemente werden im Grid autoplatziert.

```
1 .main-grid {
2   display: grid;
3   grid-template-columns: repeat(5, 1fr);
4   gap: 0.625rem;
5 }
6 .subgrid-container {
7   grid-column: span 3;
8   display: grid;
9   grid-template-columns: subgrid;
10  background-color: rgb(224 224 224);
11  border: 0.2rem dashed rgb(102 102 102);
12  padding: 0;
13 }
14 .sub-item {
15   background-color: rgb(255 85 85);
16   color: white;
17   padding: 0.625rem;
18   text-align: center;
19   font-weight: bold;
20 }
```

Benennung von Rasterlinien und Bereichen

Für einfache Anordnungen im Grid reicht Auto-Placement. Oft sollen jedoch Elemente mehrere Grid-Cells, d. h. größere Bereiche einnehmen. Um nun festzulegen, wie viele Zellen ein Element später einnimmt (siehe Abschnitt 5.2.3), wird angegeben, zwischen welchen Rasterlinien es liegen soll. Diese werden von links nach rechts bzw. von oben nach unten einfach durchnummeriert. Um hier nicht auf die Zahlen angewiesen zu sein, können bei der Definition des Rasters auch Namen vergeben werden. Diese werden in eckige Klammern vor und nach die jeweiligen Spalten- und Zeilenangaben gestellt, mit Leerzeichen können für eine Linie auch mehrere Namen vergeben werden. `grid-template-columns: [col1-start] 1fr [col1-end col2-start] 1fr`

[col2-end] definiert etwa für zwei Spalten die drei Rasterlinien als col1-start (1. Linie), col1-end und col2-start (2. Linie) und col2-end (3. Linie).

Alternativ können Elemente über benannte Bereiche anstatt zwischen Grid Lines platziert werden. Um solche Grid Areas zu definieren, sind spezielle Angaben über **grid-template-areas** möglich. Diese Eigenschaft akzeptiert mehrere, durch Leerzeichen getrennte Zeichenketten. Eine Zeichenkette (Text zwischen zwei Anführungszeichen) steht für den Inhalt einer Zeile, innerhalb der Zeichenketten werden die Namen der Rasterbereiche angegeben, wie sich diese über die Spalten hinweg aufteilen. **grid-template-areas: "header header" "nav main" "footer footer"** etwa setzt den Bereich **header** in der gesamten ersten Zeile (zwei Spalten) fest, die zweite Zeile wird in **nav** und **main** geteilt, die dritte Zeile ist als Bereich **footer** (wieder über zwei Spalten) gekennzeichnet.

Um den Steg (Gutter), also den Abstand zwischen den Rasterzellen zu definieren, kommt (wie bei Flexbox) die Eigenschaft **gap** zum Einsatz. **gap: 10px** erzeugt einen Abstand von 10 Pixeln zwischen Zeilen und Spalten, **gap: 10px 15px** erzeugt 10 Pixel Abstand zwischen den Zeilen und 15 Pixel Abstand zwischen den Spalten. Dies kann auch jeweils einzeln über die Angaben **row-gap** und **column-gap** definiert werden.

Ausrichtung von Inhalten

Mit den Angaben **justify-items** und **align-items** können über die Werte **start**, **end**, **center** und **stretch** die Zelleninhalte sowohl horizontal als auch vertikal ausgerichtet werden. Mit **justify-self** und **align-self** kann dies für einzelne Zellen überschrieben werden. **justify-content** und **align-content** besitzen dieselben Werte, richten aber nicht den Inhalt der Zellen sondern das Grid innerhalb des Containers aus, falls dieser größer als der Inhalt sein sollte.

Als Beispiel soll ein einfaches, zweispaltiges Layout dienen. Als Container wird ein `<div>`-Element verwendet, darin befinden sich die üblichen Elemente für Header, Navigation, Hauptbereich und Footer.

```

1 <body>
2   <div id="container">
3     <header>
4       <h1>Überschrift</h1>
5     </header>
6     <nav>
7       <ul>
8         <li>Navigation 1</li>
9         <li>Navigation 2</li>
10      </ul>
11    </nav>
12    <main>Lorem ipsum...</main>
13    <footer>&copy; 2025 Der*Die Autor*in</footer>
14  </div>
15 </body>

```

Zunächst werden die Eigenschaften für den Container gesetzt. Definiert werden zwei Spalten und drei Zeilen, wobei die erste Spalte ein Viertel des Platzes (**1fr**) und die zweite Spalte dreiviertel (**3fr**) erhält. Die erste und dritte Zeile (Header und Footer) sind mit einer fixen Höhenangabe gesetzt, der verbleibende Platz wird mittels **1fr**-Angabe flexibel gehandhabt.

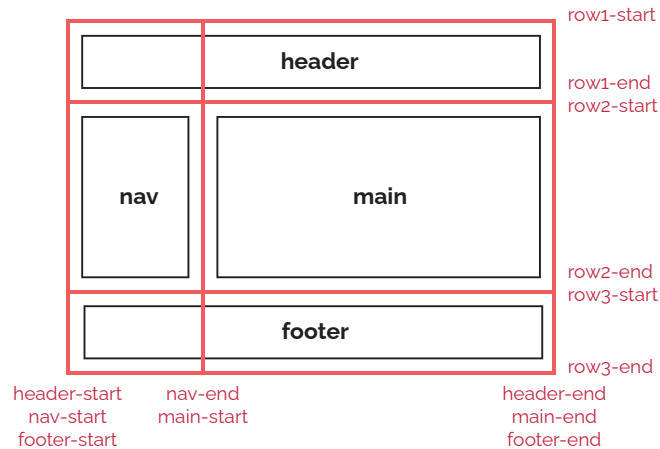


Abbildung 5.4: Das Layout mit seinen Rasterlinien (rot) und den benannten Grid-Bereichen.

In den Angaben für die Spalten und Reihen sind auch die Rasterlinien benannt, zusätzlich werden Template Areas („header“, „nav“, „main“ und „footer“) definiert. Ebenso wird ein Steg definiert.

```

1 div#container {
2   display: grid;
3   max-width: 75rem;
4   margin: 0 auto;
5   grid-template-columns:
6     [header-start nav-start footer-start] 1fr
7     [nav-end main-start] 3fr
8     [header-end main-end footer-end];
9   grid-template-rows:
10    [row1-start] 6rem
11    [row1-end row2-start] 1fr
12    [row2-end row3-start] 6rem
13    [row3-end];
14   grid-template-areas: "header header"
15                        "nav main"
16                        "footer footer";
17   gap: 0.5rem;
18 }
```

Abbildung 5.4 stellt dieses Layout mit seinen Bezeichnungen schematisch dar.

5.2.3 Elemente im Raster anordnen

Nachdem im Container nun das Layout des Grids (das Template-Grid) mit seinen Abmaßen und auch Bezeichnungen (Grid Lines oder alternativ Grid Areas) festgelegt wurden, müssen nun konkrete HTML-Elemente den einzelnen Bereichen zugewiesen werden.

Dies kann auf zwei Arten geschehen: entweder werden mittels **grid-column** und **grid-row** die Start- und End-Rasterlinien für Spalten und Zeilen (entweder über Nummern oder die vergebenen Namen) definiert oder mittels **grid-area** der vordefinierte Bereich im Raster angegeben.

Im Beispiel werden beide Arten verwendet, um dies zu demonstrieren. Im Produktionseinsatz sollte zwischen einer der beiden Methoden entschieden werden, denn beide bewirken dasselbe und sind redundant.

```
1 header {
2   grid-column: header-start / header-end;
3   grid-row: row1-start / row1-end;
4   grid-area: header;
5   background-color: rgb(250 94 94);
6 }
7 nav {
8   grid-column: nav-start / nav-end;
9   grid-row: row2-start / row2-end;
10  grid-area: nav;
11  background-color: rgb(255 132 0);
12 }
13 main {
14   grid-column: main-start / main-end;
15   grid-row: row2-start / row2-end;
16   grid-area: main;
17   color: rgb(67 67 67);
18   background-color: rgb(255 182 0);
19 }
20 footer {
21   grid-column: footer-start / footer-end;
22   grid-row: row3-start / row3-end;
23   grid-area: footer;
24   background-color: rgb(67 67 67);
25 }
```

Auch beim Grid Layout können Media Queries eingesetzt werden, um auf unterschiedliche Ausgabegeräte zu reagieren. Im Beispiel werden unter 640 Pixeln Breite etwa Navigation und Hauptbereich untereinander angeordnet. Dazu werden im Container vier Reihen definiert und die Angabe der Start- und End-Rasterlinien für Navigation, Hauptbereich und Footer entsprechend angepasst.

```
1 @media all and (max-width: 640px) {
2   div#container {
3     grid-template-rows:
4       [row1-start] 6rem
5       [row1-end row2-start] auto
6       [row2-end row3-start] auto
7       [row3-end row4-start] 6rem
8       [row4-end];
9   }
10  nav {
11    grid-column: nav-start / header-end;
12  }
13  main {
14    grid-column: header-start / main-end;
15    grid-row: row3-start / row3-end;
16  }
17  footer {
18    grid-row: row4-start / row4-end;
19  }
20 }
```

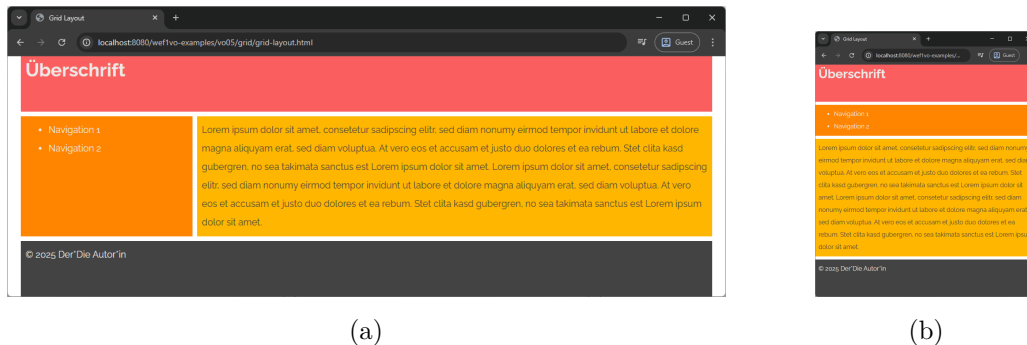


Abbildung 5.5: Ist das Browserfenster breiter als 640 Pixel, kommt die normale Formattierung zum Einsatz und die drei Spalten werden nebeneinander angeordnet (a). Wird die Breite auf unter 640 Pixel reduziert, werden die Spalten untereinander angeordnet (b).

Würde auf die Neudefinition von `grid-template-rows` verzichtet, so würde dennoch eine vierte Reihe generiert werden. Diese sogenannten *impliziten Grid Tracks* können über die Angaben `grid-auto-columns` und `grid-auto-rows` gesteuert werden. `grid-auto-rows: 100px` würde etwa jede implizit generierte Grid-Reihe mit einer Höhe von 100 Pixeln erzeugen.

Abbildung 5.5 zeigt das erstellte Layout in voller und reduzierter Breite.

Eine sehr gute Einführung zum Thema Grid Layout mit allen Begriffen und bildlichen Erklärungen bietet [6]. Ein interaktives Tutorial mit Erklärung praktischer, für das Grid ausgelegter Tools, bietet [8].

5.3 CSS-Frameworks: Bootstrap

Bevor 2014 mit dem Flexible Box Layout (Abschnitt 5.1) und 2017 dem Grid Layout (Abschnitt 5.2) zwei offizielle, von der W3C CSS Working Group entwickelte Standards zur Verfügung standen, um Layouts zu erstellen, existierten schon länger diverse Layout-Frameworks, auch einfach *CSS-Frameworks* genannt. Diese Frameworks arbeiten mit den in CSS verfügbaren Mitteln und bieten vordefinierte Stylesheets, mit deren Hilfe ohne großen Aufwand ein standardkonformes Layout erzeugt werden kann.

Die Anordnung der einzelnen Bereiche (z. B. Header, Footer, Navigation und Inhalt) erfolgt bei fast allen dieser Frameworks mit Hilfe eines Grids. Dieser Raster besteht aus einer Reihe von Spalten, die über die Gesamtbreite eines Containers aufgeteilt sind. Meist kommen dabei 12 Spalten zum Einsatz, aber auch 16 oder 24 sind möglich. Abbildung 5.6 zeigt schematisch einen solchen 12-spaltigen Raster am Beispiel von *Bootstrap*, welches im folgenden Abschnitt näher erklärt wird.

5.3.1 Bootstrap allgemein

Das Framework *Bootstrap*¹, aktuell in Version 5.3.8 verfügbar, wurde 2011 von Twitter (mittlerweile dem Bootstrap Team) entwickelt und gehört zu den populärsten CSS-

¹<https://getbootstrap.com/>

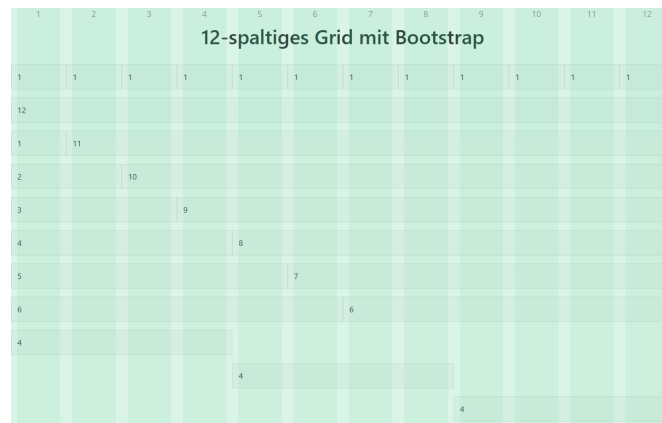


Abbildung 5.6: Boxen, angeordnet in einem 12-spaltigen Grid-System. Das farbige Overlay wird mit Hilfe einer Browser-Extension erzeugt und dient nur zur Visualisierung der Spalten.

Frameworks. Es arbeitet mit einem 12-spaltigen Rastersystem, ist responsive und verfolgt dabei einen mobile-first Ansatz. Seit Version 4 des Frameworks werden die Elemente am Raster mit Hilfe des Flexible Box Layout Moduls (Flexbox) angeordnet, in Version 3 kamen dafür noch Floats zum Einsatz. Seit Version 5.1.0 kann experimentell auch ein Modus aktiviert werden, der das CSS Grid verwendet.

Neben dem Grid-System bietet das Framework jedoch auch konsistente Lösungen für alle anderen Bereiche einer Webseite. So können Bilder oder Tabellen responsive gestaltet werden, Formularfelder ausgerichtet und gestyled werden oder User Interface Elemente wie Karusselle, Akkordeons oder Tab-Leisten erstellt werden. Für viele dieser Funktionalitäten werden dabei jedoch JavaScript-Plugins benötigt, die in Bootstrap enthalten sind. Eine Übersicht, welche Funktionalitäten Bootstrap in Detail bietet, findet sich in der offiziellen Dokumentation [11].

5.3.2 Bootstrap einbinden

Um mit Bootstrap zu arbeiten, müssen zunächst die CSS-Dateien, die das Framework zur Verfügung stellt, heruntergeladen² werden. Dies sind die so genannten kompilierten CSS- und JS- (JavaScript) Dateien, die so direkt verwendet werden können. In der Zip-Datei befinden sich die Ordner `css` und `js`, die ins eigene Projekt entpackt werden. Werden keine JavaScript-Funktionalitäten benötigt (weil etwa nur das Grid-System verwendet wird), kann auf den Ordner `js` auch verzichtet werden.

Nun wird die Bootstrap-Hauptdatei als Stylesheet in der eigenen HTML-Datei im `<head>` vor eventuellen eigenen CSS-Dateien eingebunden:

```
1 <link rel="stylesheet" href="css/bootstrap.css">
```

Um im Produktivbetrieb weniger Overhead zu haben, existiert auch eine minifizierte Version, die statt ca. 273 KB nur 226 KB groß ist. Sie wird so eingebunden:

```
1 <link rel="stylesheet" href="css/bootstrap.min.css">
```

²Für Version 5.3.x unter <https://getbootstrap.com/docs/5.3/getting-started/download/>

Möchte man nur das Bootstrap-Grid-System verwenden, aber keine von Bootstrap mitgebrachten anderen Formatierungen einsetzen, kann `bootstrap-grid.css` (bzw. `bootstrap-grid.min.css`) verwendet werden.

```
1 <link rel="stylesheet" href="css/bootstrap-grid.css">
```

Wer die Bootstrap-Dateien nicht selbst im Projekt integrieren möchte, kann auf ein so genanntes Content Delivery Network (kurz CDN) zurückgreifen. Hier findet sich die aktuellste Version des Frameworks online. Der Vorteil hierbei liegt darin, dass dieses CDN von vielen Seiten verwendet wird. Wurde im eigenen Browser zuvor bereits eine Bootstrap-Seite besucht, hat der Browser Bootstrap bereits im Cache und die Dateien müssen nicht erneut geladen werden. Der Nachteil ist, dass nicht offline entwickelt werden kann, da die Dateien auf einem Server liegen und eine Internetverbindung bestehen muss. Um Bootstrap 5.3.8 einzubinden, wird folgendes Markup benötigt (die aktuellste Version findet sich in der Dokumentation [11]).

```
1 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
  rel="stylesheet" integrity="sha384-sR114kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIA
  vNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
```

Die Angaben für `integrity` und `crossorigin` ermöglichen das so genannte Sub-resource Integrity Feature und stellen sicher, dass der Code im CDN nicht böswillig verändert wurde.

Schließlich können auch die Quelldateien von Bootstrap heruntergeladen und selbst kompiliert werden. Diese liegen in der Stylesheet-Sprache Sass vor und benötigen eine Reihe von Tools wie einen Compiler und einen Autoprefixer. Dies bietet die größte Flexibilität, setzt jedoch erweitertes Wissen voraus. Vorlesung 11 setzt sich mit diesen Workflows auseinander und gibt einen ersten Einblick.

5.3.3 Das Bootstrap Grid verwenden

Um das 12-spaltige Grid-System von Bootstrap zu verwenden, wird zunächst ein Container-Element benötigt, das dieses Grid beinhaltet. In diesem Grid werden dann Zeilen definiert, in denen dann die gewünschte Anzahl Spalten definiert werden. All dies geschieht, indem von Bootstrap vorgegebene CSS-Klassen zu HTML-Elementen hinzugefügt werden.

Bootstrap Container

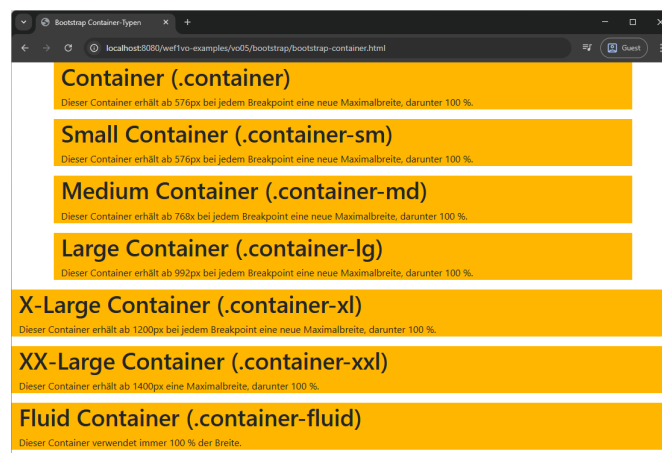
Bootstrap bietet drei unterschiedliche Container-Klassen an.

- `.container`: Setzt eine Maximalbreite (`max-width`) bei jedem responsive Breakpoint.
- `.container-fluid`: Definiert `width: 100%` bei allen Breakpoints.
- `.container-{breakpoint}`: Setzt `width: 100%` bis zum angegebenen Breakpoint und eine Maximalbreite ab diesem und bei allen weiteren.

Bootstrap definiert dabei 6 *Breakpoints*, diese sind X-Small (XS), Small (SM), Medium (MD), Large (LG), X-Large (XL) und XX-Large (XXL). Tabelle 5.1 gibt eine Übersicht über die Breakpoints und die verwendeten Breiten.

Tabelle 5.1: Bootstrap Breakpoints und die dadurch entstehenden Containerbreiten.

Klasse Breakpoint	X-Small < 576px	Small ≥ 576px	Medium ≥ 768px	Large ≥ 992px	X-Large ≥ 1200px	XX-Large ≥ 1400px
<code>.container</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-sm</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-md</code>	100%	100%	720px	960px	1140px	1320px
<code>.container-lg</code>	100%	100%	100%	960px	1140px	1320px
<code>.container-xl</code>	100%	100%	100%	100%	1140px	1320px
<code>.container-xxl</code>	100%	100%	100%	100%	100%	1320px
<code>.container-fluid</code>	100%	100%	100%	100%	100%	100%

**Abbildung 5.7:** Die Bootstrap Container-Typen bei einer Breite von 1.100 Pixel. Die ersten vier haben eine Maximalbreite von 960 Pixel, die letzten drei nehmen 100 % der Breite ein.

Wird etwa die Klasse `.container` verwendet, nimmt der Container 100 % der Viewport breite bis 575 Pixel ein. Zwischen 576 und 767 Pixel ist der Container dann 540 Pixel breit, zwischen 768 und 991 Pixel ist er 720 Pixel breit. Zwischen 992 und 1.199 Pixel ist die Breite 960 Pixel, zwischen 1.200 und 1.399 Pixel ist er 1.140 Pixel breit und ab 1.400 Pixel ist bleibt der Container bei einer Breite von 1.320 Pixel.

Abbildung 5.7 zeigt alle Containertypen bei einer Viewport-Breite von 1.100 Pixel. Die Klassen `.container`, `.container-sm`, `.container-md` und `.container-lg` sind 960 Pixel breit, die Klassen `.container-xl`, `.container-xxl` und `.container-fluid` nehmen 100 % der Breite ein.

Bootstrap Spalten am Grid erstellen

In diesen Containern können nun Spalten im Zwölferraster eingefügt werden. Spalten müssen immer ein einer Zeile mit der Klasse `.row` eingefügt werden.

Darin werden Klassen, die mit `.col-` beginnen, eingefügt. `.col-1` ist eine Spalte, die sich über eine Spalte des Rasters erstreckt, `.col-6` über 6 Spalten und `.col-12` belegt alle 12 Spalten, nimmt also die gesamte Breite ein. In einer Reihe sollten Spaltenzahlen

Tabelle 5.2: Bootstrap Breakpoints für Spaltenangaben.

Feature Breakpoint	X-Small < 576px	Small ≥ 576px	Medium ≥ 768px	Large ≥ 992px	X-Large ≥ 1200px	XX-Large ≥ 1400px
Containerbreite Klassenpräfix	Auto .col-	540px .col-sm-	720px .col-md-	960px .col-lg-	1140px .col-xl-	1320px .col-xxl-

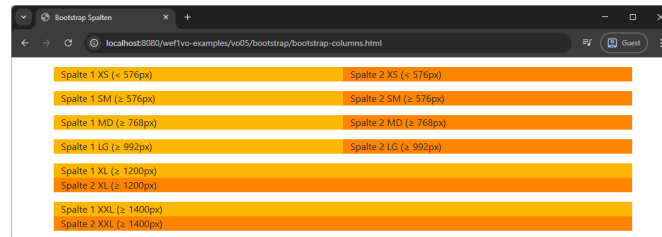


Abbildung 5.8: Die Bootstrap Spalten-Typen bei einer Breite von 1.100 Pixel. Die ersten vier sind nebeneinander angeordnet, weil die notwendige Minimalbreite erreicht ist, die letzten beiden sind untereinander mit voller Breite angeordnet, weil die Breakpoints noch nicht erreicht sind.

möglichst auf 12 addieren. Der folgende Code zeigt eine Zeile mit zwei gleich verteilten Spalten:

```

1 <div class="container">
2   <div class="row">
3     <div class="col-6">Spalte 1</div>
4     <div class="col-6">Spalte 2</div>
5   </div>
6 </div>

```

Weiters gibt es auch hier Spaltenklassen, die auf die schon beim Container gezeigten Breakpoints reagieren. Ab diesem Breakpoint nehmen die Spalten die angegebene Breite ein, darunter 100 %. Tabelle 5.2 zeigt die Breakpoints der Spalten.

Werden Spalten mit `.col-` angelegt, so bleiben diese auch beim kleinsten Container immer nebeneinander. Spalten mit `.col-sm-` stehen ab 576 Pixel nebeneinander, Spalten mit `.col-md-` stehen ab 768 Pixel nebeneinander, Spalten mit `.col-lg-` ab 992 Pixel, Spalten mit `.col-xl-` ab 1.200 Pixel und Spalten mit `.col-xxl-` ab 1.400 Pixel.

Abbildung 5.8 zeigt alle Spaltentypen in einer 2-spaltigen Anordnung bei einer Viewportbreite von 1.100 Pixel. Die Klassen `.col-6`, `.col-sm-6`, `.col-md-6` und `.col-lg-6` werden nebeneinander angeordnet, `.col-xl-6` und `.col-xxl-6` untereinander, weil die Breakpoints von 1.200 und 1.400 Pixel bei 1.100 Pixel noch nicht erreicht sind.

Diese verschiedenen Spaltenklassen können und sollen auch miteinander kombiniert werden. So können responsive Layouts einfach erreicht werden. Der folgende Code zeigt ein Layout mit drei Spalten ab dem Large-Breakpoint von 992 Pixel. Darunter – zwischen 991 und 768 Pixel – greifen Medium-Spalten. D.h. hier entsteht ein Layout in dem die ersten beiden Spalten nebeneinander und die dritte darunter liegt. Unter 767 Pixel liegen alle drei Spalten untereinander.

```

1 <div class="container">

```

```

2 <div class="row">
3   <div class="col-md-4 col-lg-3">Spalte 1</div>
4   <div class="col-md-8 col-lg-6">Spalte 2</div>
5   <div class="col-md-12 col-lg-3">Spalte 3</div>
6 </div>
7 </div>

```

In diesem Fall addieren die Angaben für Medium und Large immer auf 12. Es ist jedoch auch möglich, in einer Zeile wesentlich mehr Spalten unterzubringen, um noch flexibler agieren zu können.

```

1 <div class="container">
2   <div class="row">
3     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 1</div>
4     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 2</div>
5     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 3</div>
6     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 4</div>
7     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 5</div>
8     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 6</div>
9     <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 7</div>
10    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 8</div>
11    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 9</div>
12    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 10</div>
13    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 11</div>
14    <div class="col-md-6 col-xl-4 col-xxl-3">Spalte 12</div>
15  </div>
16 </div>

```

In diesem Beispiel werden ab 1.400 Pixel 4 Spalten erzeugt, zwischen 1.200 und 1.399 Pixel 3 Spalten, zwischen 768 und 1.199 Pixel 2 Spalten und alles darunter wird in einer Spalte angeordnet.

Zweispaltiges Layout mit Bootstrap

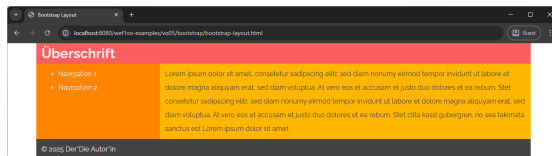
Das zweispaltige Layout, das bereits in Abschnitt 5.2 erstellt wurde, soll nun auch mit Bootstrap realisiert werden. Das HTML-Markup sieht wie folgt aus:

```

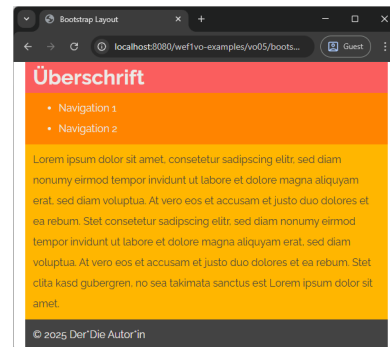
1 <body>
2   <div class="container">
3     <div class="row">
4       <header class="col-md-12">
5         <h1>Überschrift</h1>
6       </header>
7       <nav class="col-md-3">
8         <ul>
9           <li>Navigation 1</li>
10          <li>Navigation 2</li>
11        </ul>
12      </nav>
13      <main class="col-md-9">Lorem ipsum...</main>
14      <footer class="col-md-12">&copy; 2025 Der*Die Autor*in</footer>
15    </div>
16  </div>
17 </body>

```

Verwendet werden die klassischen Strukturelemente, zusätzlich kommt noch ein `<div>` als Zeile ins Spiel. Die Spaltenklassen werden bei den Elementen hinzugefügt.



(a)



(b)

Abbildung 5.9: Ist das Browserfenster breiter als 768 Pixel, kommt die normale Formattierung zum Einsatz und die drei Spalten werden nebeneinander angeordnet (a). Wird die Breite auf unter 768 Pixel reduziert, werden die Spalten untereinander angeordnet (b).

In diesem Fall wird auf die Medium-Klassen zurückgegriffen, somit werden Navigation und Hauptbereich ab 768 Pixel nebeneinander und darunter untereinander angeordnet. Abbildung 5.9 stellt beide Fälle dar.

5.3.4 Zusätzliche Bootstrap Funktionalitäten

Neben den soeben erwähnten, grundsätzlichen Grid-Funktionalitäten bietet Bootstrap noch einige weitere Dinge, die eingesetzt werden können. Von *Gutter-Klassen*, die im Grid verwendet werden können, um Abstände zwischen den Spalten (Gutter) zu regeln (`.gx-` und `.gy-`) bis hin zu *Utility-Klassen* um Margin und Padding anzupassen (`.m-` und `.p-`), Hintergründe zu setzen oder schnell einen Rahmen um ein Element zu erstellen.

Die Bootstrap-Dokumentation [11] listet all diese und auch noch darüber hinausgehende Funktionalitäten auf.

5.3.5 Weitere CSS-Frameworks

Neben Bootstrap existieren noch eine Vielzahl anderer Frameworks. Aktuelle Vertreter sind responsive, auch Konzepte wie *mobile-first* werden von diesen Frameworks abgedeckt. Der Vollständigkeit halber ist auch das erste bekannte Grid-Framework aufgelistet.

960.gs und Unsemantic

Das *960 Grid System*³ (kurz 960.gs) ist quasi der Urvater aller Grid-basierten Layout-Frameworks. Entstanden 2009 bietet es die Möglichkeit, ein Layout in 12, 16 oder auch 24 Spalten zu gestalten. Das Framework ist statisch, d. h. nicht responsive und liegt auch in einfachem CSS vor. JavaScript wird nicht benötigt. Dies macht es zum idea-

³<https://960.gs/>

len Einsteiger*innen-Framework, um sich mit der Arbeitsweise eines Grid-Frameworks vertraut zu machen.

Der Nachfolger des 960.gs ist *Unsemantic*⁴, ein ebenfalls sehr einfaches, aber bereits responsive ausgelegtes Layout-Framework. Es versteht sich als sogenanntes „Fluid Grid System“ und definiert keine fixe Anzahl an Spalten. Vielmehr können Spalten in Prozentwerten angegeben werden, dabei enthält das Framework Klassen in 5-Prozent-Abständen, also für 5, 10, 15 etc. Prozent. Darüber hinaus gibt es noch Klassen für 33 und 66 Prozent, um die Seite dritteln zu können. Die Gesamtsumme der Spalten muss immer 100 Prozent ergeben. Auch hier liegt der Fokus auf Einfachheit. Unsemantic stellt die grundlegenden Layoutfunktionalitäten zur Verfügung, der Rest kann selbst angepasst werden.

Beide Frameworks verwenden Floats um Elemente nebeneinander in Spalten anzuordnen. Sie sind aufgrund ihres Alters jedoch kaum mehr im Einsatz.

Foundation

Das CSS-Framework *Foundation*⁵ existiert wie Bootstrap seit dem Jahr 2011 und basiert ebenfalls auf einem 12er-Grid. Das Framework ist ebenso responsive, als Stylesheet-Sprache kommt Sass zum Einsatz. Flexbox ist für die Grid-Anordnung in Version 6 verantwortlich. Ebenso wird auf JavaScript für zusätzliche Funktionalitäten zurückgegriffen.

Tailwind CSS

*Tailwind CSS*⁶ geht einen etwas anderen Weg als Bootstrap und Foundation. Es gehört zur Kategorie der sogenannten Utility-Frameworks. Es stellt eine Reihe von Utility-Klassen zur Verfügung, mit dem Layouts völlig modular zusammengebaut werden können. Oft verwendete Utilities können zu Komponenten zusammengefasst werden. Tailwind funktioniert mit jeder Stylesheet-Sprache, das Konzept ist jedoch, wenig CSS selbst zu schreiben, sondern die vorgegebenen Klassen in HTML anzuwenden. Zusätzliche Konfiguration erfolgt in JavaScript. Dieses ist auch essentiell zur Installation.

5.4 CSS Custom Properties

Bei der Arbeit mit Layouts, aber prinzipiell, wann immer verschiedene Werte in Stylesheets öfters verwendet werden, stellt sich die Frage der Codewiederholung. Etwa wenn bestimmte Abstände oder Farbwerte an mehreren Stellen benötigt werden, ergibt sich schnell eine Änderungsfeindlichkeit, wenn diese Werte angepasst werden müssen. Dies muss dann unweigerlich an mehreren Stellen geschehen. *CSS Custom Properties*, auch gerne CSS Variablen genannt, schaffen hier Abhilfe.

⁴<https://unsemantic.com/>

⁵<https://get.foundation/>

⁶<https://tailwindcss.com/>

5.4.1 Anlegen von Custom Properties

Ein Custom Property muss immer innerhalb eines Selektors angelegt werden und beginnt mit zwei Bindestrichen (--).

```
1 body {  
2   --bg-color: rgb(255 182 0);  
3 }
```

Hier wird der Farbwert `rgb(255 182 0)` im Custom Property `--bg-color` definiert. Es gilt im gesamten `<body>` Element und wird auch in alle darin enthaltenen Elemente weitervererbt. Sollen also Custom Properties überall im CSS gültig sein, empfiehlt es sich, diese möglichst weit oben in der HTML-Hierarchie zu definieren. Hierfür existiert auch die Pseudoklasse `:root`, die mit dem HTML-Element gleichzusetzen ist und oft verwendet wird, um CSS Custom Properties „ganz oben“ zu definieren.

```
1 :root {  
2   --bg-color: rgb(255 182 0);  
3 }
```

Custom Properties müssen keineswegs immer einen so großen Gültigkeitsbereich aufweisen. Wird ein Wert etwa nur in Absätzen benötigt, kann er auch nur für diese definiert werden:

```
1 p {  
2   --text-color: rgb(67 67 67);  
3 }
```

5.4.2 Custom Properties anwenden

Um nun den im Property hinterlegten Wert anzuwenden, wird der Name des Properties in der Funktion `var()` angegeben. Der folgende Aufruf setzt die im `<body>` definierte Hintergrundfarbe in allen `<div>`- und `<p>`-Elementen angewandt:

```
1 div, p {  
2   background-color: var(--bg-color);  
3 }
```

Sollte das Property aus irgendeinem Grund nicht zur Verfügung stehen, kann mit Beistrich getrennt auch ein Fallback-Wert angegeben werden:

```
1 div, p {  
2   background-color: var(--bg-color, red);  
3 }
```

Dieser ist in diesem Fall der Farbwert `red`, könnte aber auch ein weiteres Custom Property sein.

5.4.3 Die Kaskade und Vererbung ausnützen

Bei Custom Properties kommt – wie von CSS generell bekannt – die Kaskade zur Anwendung. Spezifischere Selektoren kommen also eher zum Zug, ebenso später definierte. Ebenso werden Custom Properties in der Hierarchie vererbt.

Es ist also möglich, den Wert eines Properties in einem weiteren Selektor zu ändern und damit das Ergebnis zu beeinflussen. Hier werden im `<body>`-Element zunächst einige

Properties gesetzt, etwa eine Hintergrundfarbe und eine Schriftfarbe. Die Schriftfarbe wird sogleich im gesamten `<body>` angewandt:

```
1 body {
2   --bg-color: rgb(255 182 0);
3   --text-color: rgb(236 233 230);
4   --margin: 10px;
5   --size: 200px;
6   font-family: Raleway, Arial, sans-serif;
7   color: var(--text-color);
8   display: flex;
9 }
```

Alle `<div>`- und `<p>`-Elemente bekommen dann einen die restlichen Werte gesetzt:

```
1 div, p {
2   width: var(--size);
3   height: var(--size);
4   margin: var(--margin);
5   background-color: var(--bg-color, red);
6   display: flex;
7   justify-content: center;
8   align-items: center;
9 }
```

Um nun aber allen `<div>`-Elementen eine andere Hintergrundfarbe zu setzen als in dieser Formatierung vorgeschlagen, kann einfach das Custom Property `--bg-color` für alle `<div>`-Elemente überschrieben werden. Somit erhalten sie diese Farbe, ohne die gesamte `background-color`-Eigenschaft neu setzen zu müssen.

```
1 div {
2   --bg-color: rgb(250 94 94);
3 }
```

Möchte man nun die Schriftfarbe für alle `<p>`-Elemente setzen, kann auch für diese spezifisch eine neue Farbe gesetzt werden. Da aber die Schriftfarbe mit (`color`) für das `<body>`-Element gesetzt wurde, muss hier die Eigenschaft erneut angegeben werden, denn sonst wäre zwar das Custom Property neu definiert, aber es würde nicht angewendet werden (während `--bg-color` ja für alle `<div>`- und `<p>`-Elemente angewandt wurde, daher funktioniert hier einfach die Neudefinition des Properties).

```
1 p {
2   --text-color: rgb(67 67 67);
3   color: var(--text-color);
4 }
```

Abbildung 5.10 zeigt die Formatierungen angewandt auf zwei `<div>`- und ein `<p>`-Element.

CSS Custom Properties können noch viel mehr. Auch hierfür gibt es ein ausgezeichnetes Tutorial unter [2].

5.5 CSS Resets & Normalisierung

Trotz ständiger Weiterentwicklung der einzelnen Browser und auch gemeinsamer Standardisierungsbestrebungen, gibt es nach wie vor Unterschiede in der Darstellung. Dies

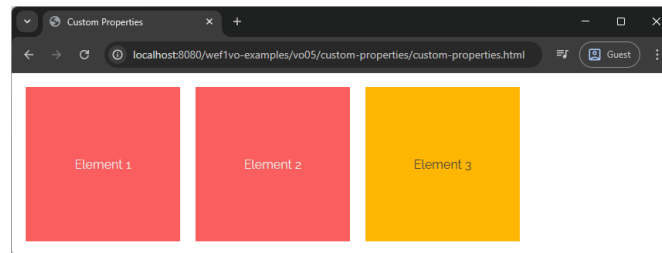


Abbildung 5.10: Die ersten beiden Boxen sind `<div>`-Elemente, die dritte ein `<p>`-Element. Höhe, Breite und Margins werden zentral gesetzt, ebenso die Hintergrundfarbe. Diese wird aber für `<div>`-Elemente nochmals überschrieben und ist daher anders. Das `<p>`-Element erhält eine andere Schriftfarbe.

liegt vor allem in verschiedenen Default-Stylesheets, die in den einzelnen Browsern hinterlegt sind. Um nun nicht auf jeden Browser Rücksicht nehmen zu müssen, also z. B. abzufragen, um welchen Browser es sich handelt und dabei bestimmte Dinge speziell anzupassen, hat es sich etabliert, die entsprechenden Styles komplett zurückzusetzen, d. h. das Browser-Stylesheet völlig außer Kraft zu setzen. Dies bietet die Möglichkeit, mit den eigenen Styles quasi von Null zu beginnen. Ein solches Rücksetzen nennt sich *CSS Reset*.

5.5.1 Alles zurücksetzen: CSS Reset

In vielen der Beispiele in dieser und der vorangehenden Vorlesung haben Sie vermutlich die folgende CSS-Angabe bemerkt:

```
1 * {
2   margin: 0;
3 }
```

Sie ist die einfachste Form eines CSS Resets und entfernt für jedes HTML-Element etwaige voreingestellte Außenabstände. Damit ist es jedoch kaum getan. Zusätzlich sollten Innenabstände, Rahmen, etwaige Outlines entfernt werden, die Zeilenhöhe auf einfach und die Schriftgröße auf 100 % gesetzt werden und noch einige andere Dinge.

Um dies nicht selbst erledigen zu müssen, haben sich im Web verschiedene, sogenannte Reset Stylesheets etabliert. Das historisch bekannteste ist *Reset CSS* von Eric Meyer [7]. Es nimmt die folgenden Einstellungen vor, diese sollen laut dem Autor jedoch primär als Startpunkt gesehen und nach Belieben angepasst werden:

```
1 /* http://meyerweb.com/eric/tools/css/reset/
2    v2.0 | 20110126
3    License: none (public domain)
4 */
5
6 html, body, div, span, applet, object, iframe,
7 h1, h2, h3, h4, h5, h6, p, blockquote, pre,
8 a, abbr, acronym, address, big, cite, code,
9 del, dfn, em, img, ins, kbd, q, s, samp,
10 small, strike, strong, sub, sup, tt, var,
11 b, u, i, center,
12 dl, dt, dd, ol, ul, li,
```

```

13 fieldset, form, label, legend,
14 table, caption, tbody, tfoot, thead, tr, th, td,
15 article, aside, canvas, details, embed,
16 figure, figcaption, footer, header, hgroup,
17 menu, nav, output, ruby, section, summary,
18 time, mark, audio, video {
19     margin: 0;
20     padding: 0;
21     border: 0;
22     font-size: 100%;
23     font: inherit;
24     vertical-align: baseline;
25 }
26 /* HTML5 display-role reset for older browsers */
27 article, aside, details, figcaption, figure,
28 footer, header, hgroup, menu, nav, section {
29     display: block;
30 }
31 body {
32     line-height: 1;
33 }
34 ol, ul {
35     list-style: none;
36 }
37 blockquote, q {
38     quotes: none;
39 }
40 blockquote:before, blockquote:after,
41 q:before, q:after {
42     content: '';
43     content: none;
44 }
45 table {
46     border-collapse: collapse;
47     border-spacing: 0;
48 }

```

Ein modernerer Ansatz für einen CSS-Reset wird von Bell in [1] beschrieben.

5.5.2 Sinnvoll zurücksetzen: CSS Normalisierung

CSS Resets stellen einen sehr radikalen Eingriff in das Renderverhalten der Webbrowser dar. Denn es werden nicht nur ungeliebte Einstellungen entfernt, sondern durchaus auch sinnvolle, wie etwa die Einrückung bei Aufzählungen. Diese müssen dann in der Regel im eigenen Stylesheet wieder definiert werden, was durchaus nicht unumstritten ist.

Aus diesem Grunde wird in vielen Webprojekten und Frameworks (wie etwas auch in Bootstrap) kein Reset Stylesheet verwendet, sondern auf *CSS Normalisierung* gesetzt. Bei diesem Ansatz werden nur Dinge korrigiert, die tatsächlich in Browsern unterschiedlich sind und zu Darstellungsproblemen führen können.

Das bekannteste Normalisierungs-Stylesheet ist *Normalize.css* von Nicolas Gallagher [5]. Es kann, nachdem es heruntergeladen wurde, auf zwei Weisen verwendet werden:

- Als Ausgangspunkt für das eigene Stylesheet. Die Inhalte von `normalize.css` werden als Basis angesehen, eigene Styles werden hinzugefügt bzw. die Ausgangsbasis angepasst.

- Als zusätzliches Stylesheet: `normalize.css` wird vor dem eigenen Stylesheet unverändert eingebunden, die eigenen Styles überschreiben – wo nötig – das Normalisierungs-Stylesheet.

Da die CSS-Datei sehr umfangreich ist, wird auf eine Angabe im Skriptum verzichtet. Unter [5] kann der Quellcode angesehen bzw. heruntergeladen werden. Da das Projekt seit 2018 jedoch nicht mehr weiterentwickelt wird, haben sich daraus zwei Abspaltungen (engl. „forks“) gebildet, die auf Normalize.css basieren, jedoch noch aktiv entwickelt werden:

- `@csstools/normalize.css` [4]: Vom Co-Autor vom ursprünglichen Normalize.css, Jonathan Neal, entwickelt und nach Auffassungsunterschieden mit Nicolas Gallagher selbstständig fortgeführt.
- `modern-normalize` [10]: Eine auf Normalize.css basierende Variante, die jedoch nur mehr moderne Browser berücksichtigt.

Normalize.css findet sich wie bereits erwähnt auch in aktuellen Bootstrap-Versionen. Ein Fork wird dort als *Bootstrap Reboot*⁷ bezeichnet.

Neben Normalize.css und seinen Forks hat sich *CSS Remedy*⁸ als moderner Ansatz etabliert. Es verfolgt das Ziel, die sinnvollen Standardformatierungen der Browser beizubehalten, aber problematische Unterschiede zu beheben.

5.6 CSS Methodiken

Resets und Normalisierung bilden eine die *technische Grundlage*, um mit konsistenten Ausgangswerten in allen Browsern zu arbeiten. Sobald ein Projekt jedoch größer wird oder Layouts bzw. Komponenten wiederverwendbar sein sollen, stellt sich zusätzlich die Frage, *wie* Styles sinnvoll strukturiert und organisiert werden können. Genau hier setzen verschiedene *CSS-Methodiken und Architekturansätze* an.

Bei kleinen Webprojekten ist die Organisation der Stylesheets bzw. der darin enthaltenen Regeln normalerweise kein großes Problem. Styles werden für den jeweiligen Anlassfall geschrieben, möglicherweise kommt ein CSS-Präprozessor zur Unterstützung hinzu (siehe Vorlesung 11). Wächst jedoch ein Projekt oder sollen Layouts bzw. Teile davon wiederverwendbar sein, so ist eine gewisse Struktur und Methodik unumgänglich. Im Laufe der Zeit haben sich einige Ansätze herauskristallisiert, wie CSS organisiert und auch modularisiert werden kann, um die Entwicklung effektiver zu gestalten.

Dieser Abschnitt soll als Anregung dienen, sich zu gegebener Zeit mit diesen Techniken auseinanderzusetzen, falls der Bedarf besteht bzw. der Eindruck entsteht, dass die eigene CSS-Codebasis einer Strukturierung bedarf.

5.6.1 Historische Architekturansätze: OOCSS, SMACSS und ITCSS

Bevor moderne Sprachfeatures wie `@layer` oder Custom Properties verfügbar waren, wurden verschiedene Architekturansätze entwickelt, um große Stylesheets besser handhabbar zu machen.

⁷<https://getbootstrap.com/docs/5.3/content/reboot/>

⁸<https://github.com/jensimmons/cssremedy>

Object Oriented CSS (OOCSS) versucht, wiederverwendbare „Objekte“ zu definieren, indem Struktur (z. B. Breiten, Abstände) und Aussehen (Farben, Hintergründe) getrennt beschrieben werden. Ziel ist es, Duplikate zu vermeiden und Komponenten möglichst unabhängig vom Kontext zu halten.

SMACSS (Scalable and Modular Architecture for CSS) ordnet Regeln in Kategorien wie *Base*, *Layout*, *Module*, *State* und *Theme* ein. Ideen daraus finden sich heute noch in vielen Codebasen wieder (z. B. die Trennung von Grundformatierungen und Komponenten).

ITCSS (Inverted Triangle CSS) organisiert Styles in einer Schichtenpyramide vom Allgemeinen zum Speziellen (z. B. *Settings* → *Tools* → *Generic* → *Elements* → *Objects* → *Components* → *Utilities*). Dieses Schichtenmodell lässt sich heute sehr gut mit `@layer` (siehe unten) direkt in CSS nachbilden.

Diese Ansätze sind aus historischer und konzeptioneller Sicht interessant, werden aber in modernen Projekten häufig durch eine Kombination aus klaren Namenskonventionen (z. B. BEM), Utility-Klassen und nativen Sprachmitteln wie `@layer` ersetzt. Eine gute Übersicht über diese klassischen Ansätze bietet <https://cathydutton.co.uk/posts/css-layers/>.

5.6.2 Block Element Modifier (BEM)

BEM ist ein Ansatz, wiederverwendbare Komponenten mit Hilfe von CSS-Klassen zu kreieren. Stile, die nach dem BEM-Prinzip erzeugt worden sind, können völlig frei in jedem Szenario wiederverwendet werden und erzeugen daher große Flexibilität.

Zu jedem der drei namensgebenden Teile gibt es eine Terminologie gepaart mit einer strikten Namenskonvention:

- **Block:** eigenständige, logische Einheit auf der Seite (z. B. Header, Formular, Menü).
- **Element:** Bestandteil eines Blocks (z. B. Button innerhalb eines Formulars).
- **Modifier:** Variation oder Zustand eines Blocks oder Elements (z. B. „dunkel“, „kompakt“).

Block

Blöcke sind eigenständig und dürfen keine Abhängigkeiten zu anderen Blöcken oder Elementen haben. Sie werden mit einer CSS-Klasse definiert (hier etwa die Suche auf einer Seite):

```
1 .site-search {
2   ...
3 }
```

Angewandt würde die Klasse beim `<form>`-Element:

```
1 <form action="..." class="site-search">
2   ...
3 </form>
```

Element

Als Element wird ein Teil eines Blocks bezeichnet, also etwa der Titel im Header, der Login-Button, ein Menü-Eintrag oder das Suchfeld.

Elemente existieren immer in Zusammenhang mit ihrem Block, haben alleine also keine Bedeutung. Wiederum wird eine CSS-Klasse definiert, als Namenskonvention wird der Name des Blocks, gefolgt von zwei Unterstrichen und dem Elementnamen gewählt:

```
1 .site-search__field {
2   ...
3 }
```

Diese Klasse würde beim Suchfeld angewandt werden (ebenso würde für den Button eine eigene Klasse entsprechend gewählt werden):

```
1 <form action="..." class="site-search">
2   <input type="search" class="site-search__field">
3   <button type="submit" class="site-search__button">Search</button>
4 </form>
```

Modifier

Modifier beschreiben Variationen eines Blocks, also ein zentrierter Header, ein verkürztes Login-Formular (ohne Labels), ein vertikales Menü oder eine anders gestylte Suche.

Modifier beziehen sich ebenfalls konkret auf einen Block. Die CSS-Klasse besteht aus dem Blocknamen, gefolgt von zwei Bindestrichen und dem Modifikator:

```
1 .site-search--full {
2   ...
3 }
```

Angewandt wird die Klasse beim Block selbst:

```
1 <form action="..." class="site-search site-search--full">
2   <input type="search" class="site-search__field">
3   <button type="submit" class="site-search__button">Search</button>
4 </form>
```

Auch Elemente können Modifikatoren haben. Diese werden dann etwa wie folgt definiert:

```
1 .site-search__button--clicked {
2   ...
3 }
```

Mehr Details zu BEM finden sich auf der Projektseite unter <http://getbem.com/>.

5.6.3 Style Tools for UI Components (SUIT CSS)

SUIT CSS ist eine Modifikation der BEM-Prinzipien, um kleine Probleme mit diesen zu lösen, so ist einerseits die Benennung der einzelnen Teile, aber vor allem die Namenskonvention anders und soll besser lesbar sein. Blocks heißen nun Components, Elements werden als Descendants bezeichnet. Neben den bekannten Modifiern gibt es auch Zustände für Komponenten.

Zusätzlich führt SUIT CSS noch Utilities ein, die innerhalb von Components verwendet werden können und oft gebrauchte Formatierungen beinhalten.

Components

Components entsprechen genau den Blöcken in BEM. Sie sollen alleinstehend funktionieren und mit jedem Element anwendbar sein. Daher werden Klassenselektoren verwendet.

Komponenten werden jedoch in Pascal Case (groß begonnen, Wortgrenzen mit Großbuchstaben) geschrieben, um sie leichter von altem, „legacy“ Code (der nicht BEM oder SUIT CSS verwendet) unterscheiden zu können. Die Suche würde in SUIT CSS wie folgt definiert:

```
1 .SiteSearch {  
2   ...  
3 }
```

Angewandt würde die Klasse beim `<form>`-Element:

```
1 <form action="..." class="SiteSearch">  
2   ...  
3 </form>
```

Descendants

Descendants entsprechen den Elementen in BEM, gehören also logisch zu einer Component. Sie werden mit einem Bindestrich vom Component-Namen getrennt und in Camel Case (klein begonnen, Wortgrenzen mit Großbuchstaben) geschrieben:

```
1 .SiteSearch-field {  
2   ...  
3 }
```

Diese Klasse würde beim Suchfeld angewandt werden, ebenso ein Descendant für den Button:

```
1 <form action="..." class="SiteSearch">  
2   <input type="search" class="SiteSearch-field">  
3   <button type="submit" class="SiteSearch-button">Search</button>  
4 </form>
```

Modifier

Modifier werden auch hier mit doppeltem Bindestrich geschrieben, auch bei ihnen kommt Camel Case zum Einsatz:

```
1 .SiteSearch--full {  
2   ...  
3 }
```

Angewandt wird die Klasse beim Block selbst:

```
1 <form action="..." class="SiteSearch SiteSearch--full">  
2   <input type="search" class="SiteSearch-field">  
3   <button type="submit" class="SiteSearch-button">Search</button>  
4 </form>
```

States

Mit States wird eine Zustandsänderung einer Komponenten bezeichnet. Die Notation beginnt mit `is-` und wird in Camel Case fortgesetzt. Diese Klassen existieren immer nur als Zusätze zu bestehenden Komponenten:

```
1 .SiteSearch {  
2   ...  
3 }  
4 .SiteSearch.is-searching {  
5   ...  
6 }
```

Angewandt würde dies wie folgt:

```
1 <form action="..." class="SiteSearch is-searching">  
2   ...  
3 </form>
```

Utilities

Utilities schließlich sind Klassen, die in Components für oft benötigte Dinge zum Einsatz kommen. Sie definieren etwa Abstände, Media Query Breakpoints oder Floats. Sie Beginnen mit dem `u-`-Präfix und werden in Camel Case geschrieben.

Das folgende Utility etwa zentriert Text innerhalb von Components:

```
1 .u-textCenter {  
2   text-align: center;  
3 }
```

Es wird z.B. in der Suche angewandt, um darin alles zentriert darzustellen:

```
1 <form action="..." class="SiteSearch u-textCenter">  
2   ...  
3 </form>
```

Mehr Details zu SUIT CSS finden sich auf der Projektseite unter <https://suitcss.github.io/>.

5.6.4 Cascade Layers (@layer)

Ergänzend zu den vorgestellten Methodiken sein nochmals *CSS Cascade Layers* explizit erwähnt, die bereits im Rahmen der Vorstellung der Kaskade in Abschnitt 3.6 vorkamen. Eingeführt 2021 in [13], sind diese seit 2022 in allen modernen Browsern unterstützt. Mit ihnen lässt sich die Reihenfolge verschiedener Stilgruppen festlegen.

Dadurch können Entwickler*innen Regeln logisch strukturieren, bevor Dinge wie Specificity und Reihenfolge schlagend werden. Damit ist **@layer** das native, moderne Gegenstück zu früheren Architekturansätzen wie SMACSS oder ITCSS und ergänzt Konzepte wie CSS Resets und Normalisierung aus Abschnitt 5.5, die heute oft in eine eigene Layer (**@layer reset**) gelegt werden.

Mit dem Schlüsselwort (auch „at-rule“ genannt) **@layer** wird zunächst eine Reihenfolge an Layern definiert. Dazu werden Layer-Namen mit Komma getrennt angegeben. Namen, die weiter hinten stehen, haben mehr Gewicht.

Mit


```
1 @layer reset, base, components, utilities;
```

werden die vier Layer `reset`, `base`, `components` und `utilities` definiert.

Um nun Regeln einem Layer zuzuweisen, können diese mit der `@layer`-Angabe und dem Namen gekapselt werden:

```
1 @layer reset {
2   *, *::before, *::after {
3     box-sizing: border-box;
4     margin: 0;
5     padding: 0;
6   }
7 }
8 @layer base {
9   body {
10    font-family: system-ui, sans-serif;
11    line-height: 1.5;
12  }
13 }
14 @layer components {
15   .button {
16     background: steelblue;
17     color: white;
18   }
19 }
20 @layer utilities {
21   .text-center {
22     text-align: center;
23   }
24 }
```

Alternativ kann auch bei einem Import ein Layer angegeben werden und so die Regeln eines gesamten Stylesheets einem Layer zugewiesen werden:

```
1 @import url("modern-normalize.css") layer(reset);
2 @import url("base.css") layer(base);
3 @import url("components.css") layer(components);
4 @import url("utilities.css") layer(utilities);
```

Styles in früher definierten Layers (z. B. `reset`) werden immer vor späteren Layers (z. B. `utilities`) berücksichtigt – unabhängig von der Reihenfolge im Stylesheet oder der Selektor-Specificity. Dies macht komplexe Stylesheets deutlich berechenbarer und erlaubt eine klare Trennung von Basisformatierungen (auch Resets oder Normalisierung) und projekt- oder komponentenspezifischen Stilen.

5.6.5 Weitere Methodiken und moderne Konzepte

Neben den erwähnten Dingen sind auch die folgenden Dinge interessant bzw. nützlich:

- **Atomic CSS:** Statt größere Komponentenklassen zu definieren, werden sehr kleine, hochgradig spezialisierte Klassen verwendet, die jeweils genau eine Eigenschaft setzen (z. B. `.mt-4` nur für den oberen Außenabstand oder `.text-center` nur für die Textausrichtung). Layouts entstehen durch das Kombinieren vieler solcher „Atome“ direkt im HTML. Eine ausführliche Diskussion des Begriffs findet sich unter <https://css-tricks.com/lets-define-exactly-atomic-css/>.

- **Utility-First CSS:** Frameworks wie Tailwind CSS setzen fast ausschließlich auf kleine, wiederverwendbare Klassen, die direkt im HTML kombiniert werden: <https://heydonworks.com/article/what-is-utility-first-css/>.
- **CSS Nesting:** Regeln können direkt innerhalb eines Selektors verschachtelt werden (z. B. `&:hover`), was die Lesbarkeit erhöht und Abhängigkeiten deutlicher macht. <https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Nesting/Using> gibt eine gute Übersicht.

5.7 Weiterführende Literatur

Zum Thema Layouts finden sich grundsätzliche Ansätze wiederum in den Standardwerken [9, 12].

Generell kann für alle erwähnten Frameworks (Bootstrap, Foundation, Tailwind CSS) primär die eigene Dokumentation empfohlen werden. Selbstverständlich finden sich zahllose Tutorials online.

Zu den Themen Flexible Box Layout, Grid Layout und Custom Properties sind die erwähnten Tutorials [2, 3, 6, 8] derzeit sicherlich die beste (freie) Referenz.

Die Themen CSS Reset, Normalisierung sowie CSS Methodiken werden in den erwähnten Online-Ressourcen am besten abgearbeitet.

Quellenverzeichnis

- [1] Andy Bell. *A (more) Modern CSS Reset*. 18. Sep. 2023. URL: <https://piccalil.li/blog/a-more-modern-css-reset/> (besucht am 21. 11. 2025) (siehe S. 5-26).
- [2] Chris Coyier. *CSS Custom Properties Guide*. 17. Okt. 2024. URL: <https://css-tricks.com/a-complete-guide-to-custom-properties/> (besucht am 21. 11. 2025) (siehe S. 5-24, 5-33).
- [3] Chris Coyier. *CSS Flexbox Layout Guide*. 12. Aug. 2024. URL: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/> (besucht am 21. 11. 2025) (siehe S. 5-6, 5-33).
- [4] csstools. *normalize.css*. Version 12.1.1. 29. Dez. 2023. URL: <https://github.com/csstools/normalize.css> (siehe S. 5-27).
- [5] Nicolas Gallagher. *Normalize.css. A modern, HTML5-ready alternative to CSS resets*. 5. Nov. 2018. URL: <https://necolas.github.io/normalize.css/> (besucht am 21. 11. 2025) (siehe S. 5-26, 5-27).
- [6] Chris House. *CSS Grid Layout Guide*. 14. Nov. 2025. URL: <https://css-tricks.com/snippets/css/complete-guide-grid/> (besucht am 21. 11. 2025) (siehe S. 5-15, 5-33).
- [7] Eric Meyer. *CSS Tools: Reset CSS*. Jan. 2011. URL: <https://meyerweb.com/eric/tools/css/reset/> (besucht am 21. 11. 2025) (siehe S. 5-25).
- [8] Mozilla. *CSS Grid PlayGround*. 27. Jan. 2018. URL: <https://mozilladevelopers.github.io/playground/css-grid/> (besucht am 21. 11. 2025) (siehe S. 5-15, 5-33).
- [9] Peter Müller. *Einstieg in HTML und CSS. Webseiten programmieren und gestalten*. 3. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Sep. 2024 (siehe S. 5-33).
- [10] Sindre Sorhus. *modern-normalize*. Version 3.0.1. 5. Sep. 2024. URL: <https://github.com/sindresorhus/modern-normalize/> (siehe S. 5-27).
- [11] The Bootstrap Team. *Bootstrap Documentation*. Version 5.3.x. 8. Nov. 2025. URL: <https://getbootstrap.com/docs/> (siehe S. 5-16, 5-17, 5-21).
- [12] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 5-33).
- [13] World Wide Web Consortium. *CSS Cascading and Inheritance Level 5*. W3C Candidate Recommendation Snapshot. 13. Jan. 2022. URL: <https://www.w3.org/TR/css-cascade-5/> (siehe S. 5-31).

- [14] World Wide Web Consortium. *CSS Flexible Box Layout Module Level 1*. W3C Candidate Recommendation Draft. 14. Okt. 2025. URL: <https://www.w3.org/TR/css-flexbox-1/> (siehe S. 5-1).
- [15] World Wide Web Consortium. *CSS Grid Layout Module Level 1*. W3C Candidate Recommendation Draft. 26. März 2025. URL: <https://www.w3.org/TR/css-grid-1/> (siehe S. 5-7).
- [16] World Wide Web Consortium. *CSS Grid Layout Module Level 2*. W3C Candidate Recommendation Draft. 26. März 2025. URL: <https://www.w3.org/TR/css-grid-2/> (siehe S. 5-10).